Вопрос №1

1) Нет ограничений для **первичного ключа** (client_id) — это нарушение сущностной целостности;

Необходимо добавить обозначение, что данное поле является первичным ключом (*PRIMARY KEY*). Ещё нужно добавить ограничение, чтобы первичный ключ никогда не мог быть пуст (*NOT NULL*).

2) В зависимости от используемой БД, необходимо либо использовать тип данных **SERIAL** (в postgresql), либо устанавливать **AUTO_INCREMENT** (в таких БД как MySQL) для **первичного ключа**;

В PostgreSQL ключевое слово **SERIAL** используется для автоматической генерации возрастающих числовых значений в столбце.

3) Не указано, что поле city_id является **внешним ключом** — это нарушение ссылочной целостности;

Необходимо указать имя связанной таблицы и имя столбца из этой таблицы, на который будет указывать внешний ключ.

4) Также для **внешнего ключа** не установлено <u>действие при удалении или изменении связанной строки из главной таблицы</u> — это будет нарушение ссылочной целостности;

По умолчанию устанавливается опция **NO ACTION,** которая выдаст **ошибку** при удалении связанной строки в главной таблице. Необходимо установить **null** (SET NULL) в поле внешнего ключа или значения по умолчанию (SET DEFAULT) при удалении записи из главной таблицы.

5) Поле ввода **возраста** лучше заменить на **дату рождения;**

Для даты рождения должны быть установлены ограничения, например, чтобы нельзя было поставить дату больше сегодняшней.

6) Не установлено значение по умолчанию для **даты регистрации** (reg_date) — это нарушение доменной целостности;

Должно быть быть установлено значение по умолчанию для столбца **reg_date** при помощи атрибута DEFAULT.

7) Не установлено ограничения уникальности для поля **login** — нарушение пользовательской (бизнес-целостности);

Поле **login** должно иметь уникальное значение, этого можно достичь, например при помощи атрибута **UNIQUE** в postgresql;

8) Поле **login** лучше переименовать в **client_username**, так как некоторые системы могут интерпретировать ***login*** как специальное слово;
9) У пользователя нет **пароля** — это нарушение пользовательской (бизнес-целостности);

Для дальнейшей аутентификации пользователя, должны иметься соответствующие данные, например логин и пароль.

10) Не установлены ограничения на **пустые поля** — нарушение доменной целостности.

В данной таблице все поля, кроме фамилии и внешнего ключа (client_surname, *city_id*), не могут быть пустыми и должны быть установлены ограничения *NOT NULL*.

Пример исправленной схемы БД:

```
clients (
     client_id          SERIAL PRIMARY KEY NOT NULL,
--уникальный id клиента
     client_name        varchar(255) NOT NULL, --имя
клиента
     client_surname     varchar(255), --фамилия клиента
     client_username        varchar(30) UNIQUE NOT NULL,
--логин, который придумал клиент
     client_password    varchar(255) NOT NULL, --пароль,
который придумал клиент (храним в виде хэша с
добавлением соли)
     FOREIGN KEY city_id              number REFERENCES
City (Id) ON DELETE SET NULL, --id города, который указал
клиент
     date_of_birth      date NOT NULL, --дата рождения
     reg_date           date DEFAULT date.now, --дата
регистрации на сайте
    CONSTRAINT valid_birth_date CHECK (date_of_birth <=
CURRENT_DATE)
)
```

Вопрос №2

Для начала необходимо изменить тип данных для стоимости товара с **number** на **decimal** (этот тип данных используется для величин, для которых необходимо сохранить повышенную точность) или другие, например, **smallmoney** (decimal(10, 4)) или **money** (decimal(19, 4)) в Transact-SQL (Microsoft SQL Server), а также money в PostgreSQL. Естественно необходимо добавить необходимые ограничения для всех полей, чтобы не было нарушений целостности.

Хранение истории изменений стоимости товара можно сделать двумя вариантами:

1) С точки зрения **реляционной алгебры** лучше хранить цену в таблице с историей:

```
items (
     item_id       SERIAL PRIMARY KEY NOT NULL,
--уникальный id товара
     item_name     varchar(255) NOT NULL, --наименование
товара
)

history_cost (
     history_id    SERIAL PRIMARY KEY UNIQUE NOT NULL,
--уникальный id записи
     FOREIGN KEY item_id     number REFERENCES items
(item_id) ON DELETE SET NULL, -- id товара
     item_cost     number NOT NULL CHECK (item_cost > 0),
--стоимость товара
     date_changed  date NOT NULL, --дата изменении цены
на товар
     CONSTRAINT valid_date_changed CHECK (date_changed <=
CURRENT_DATE)
)
```

2) С точки зрения **производительности** актуальную цену можно хранить в таблице товаров (*items*), а историю в отдельной таблице. Этот вариант оправдан только в случае, если соединение (*join*) с таблицей истории сильно влияет на производительность:

```
items (
     item_id       SERIAL PRIMARY KEY UNIQUE NOT NULL,
--уникальный id товара
     item_name     varchar(255) NOT NULL, --наименование
товара
     item_cost     number NOT NULL CHECK (item_cost > 0),
--стоимость товара
)

history_cost (
     history_id    SERIAL PRIMARY KEY UNIQUE NOT NULL,
--уникальный id записи
     FOREIGN KEY item_id     number  REFERENCES items
(item_id) ON DELETE SET NULL, -- id товара
     date_changed  date NOT NULL, --дата изменении цены
на товар
     CONSTRAINT valid_date_changed CHECK (date_changed <=
CURRENT_DATE)
)
```

Вопрос №3

```
SELECT D.DEPARTMENT_ID,
       DEPARTMENT_NAME,
       SUM(SALARY) AS TOTAL_SALARY
FROM EMPLOYEES AS E JOIN DEPARTMENTS AS D
       ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
WHERE MANAGER_ID = NULL
AND D.LOCATION_ID = (SELECT LOCATION_ID
       FROM LOCATIONS WHERE CITY = 'SEOUL')
GROUP BY D.DEPARTMENT_ID, DEPARTMENT_NAME
HAVING SUM(SALARY) > 100000;
```

Вопрос №4

Карандаш положили вплотную к стене или в комнату с карандашом не возможно попасть, поэтому и перепрыгнуть через карандаш не возможно.